

Applications of Stack

Expression Conversion, Postfix Evaluation,
Recursion and Tower of Hanoi

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Three Expression Notations

Consider the operation:

$$A + B$$

Notation	Form	Example
Infix	Operand Operator Operand	$A + B$
Prefix	Operator Operand Operand	$+AB$
Postfix	Operand Operand Operator	$AB+$

Three Expression Notations

Consider the operation:

$$A + B$$

Notation	Form	Example
Infix	Operand Operator Operand	$A + B$
Prefix	Operator Operand Operand	$+AB$
Postfix	Operand Operand Operator	$AB+$

Key Difference

The position of the operator determines the notation.

Why is Infix Difficult for a Computer?

Consider:

$$A + B * C$$

The computer must understand:

- Operator precedence
- Operator associativity
- Parentheses
- Evaluation order

Because multiplication has higher precedence:

$$A + (B * C)$$

Why is Infix Difficult for a Computer?

Consider:

$$A + B * C$$

The computer must understand:

- Operator precedence
- Operator associativity
- Parentheses
- Evaluation order

Because multiplication has higher precedence:

$$A + (B * C)$$

In postfix notation:

$$ABC * +$$

The required evaluation order is directly encoded in the expression.

Examples of Equivalent Expressions

Infix	Postfix	Prefix
$A + B$	$AB+$	$+AB$
$A + B * C$	$ABC * +$	$+A * BC$
$(A + B) * C$	$AB + C*$	$* + ABC$
$(A - B)/(C + D)$	$AB - CD + /$	$/ - AB + CD$

Examples of Equivalent Expressions

Infix	Postfix	Prefix
$A + B$	$AB+$	$+AB$
$A + B * C$	$ABC * +$	$+A * BC$
$(A + B) * C$	$AB + C*$	$* + ABC$
$(A - B)/(C + D)$	$AB - CD + /$	$/ - AB + CD$

Observation

Postfix and prefix expressions do not require parentheses to specify the evaluation order.

Operator Precedence

Operators	Precedence	Associativity
$\wedge$	Highest	Right to left
$*$ $/$ $\%$	Medium	Left to right
$+$ $-$	Lowest	Left to right

Parentheses override normal precedence.

$$A + B * C$$

means:

$$A + (B * C)$$

whereas:

$$(A + B) * C$$

forces addition first.

Role of the Stack

During infix-to-postfix conversion:

- Operands are immediately added to the output.
- Operators are temporarily stored in a stack.
- Parentheses control when operators are removed.
- Higher-precedence operators are processed first.

Role of the Stack

During infix-to-postfix conversion:

- Operands are immediately added to the output.
- Operators are temporarily stored in a stack.
- Parentheses control when operators are removed.
- Higher-precedence operators are processed first.

Important

The stack stores operators, not operands.

Conversion Rules

Scan the infix expression from left to right.

- ① If the symbol is an operand, add it to postfix.
- ② If the symbol is '(', push it.
- ③ If the symbol is ')', pop until '(' is found.
- ④ If the symbol is an operator:
 - Pop operators having greater precedence.
 - Pop equal-precedence operators for left-associative operators.
 - Push the current operator.
- ⑤ At the end, pop all remaining operators.

Algorithm: Infix to Postfix

Algorithm

- 1 Create an empty operator stack.
- 2 Scan each token from left to right.
- 3 Copy operands directly to the postfix expression.
- 4 Push opening parentheses.
- 5 On a closing parenthesis, pop until the matching opening parenthesis.
- 6 For an operator, pop all higher-priority operators.
- 7 Push the current operator.
- 8 After scanning, pop every remaining operator.

Dry Run: $A + B * C$

Convert:

$$A + B * C$$

Symbol	Action	Stack	Postfix
A	Add operand	Empty	A
$+$	Push operator	$+$	A
B	Add operand	$+$	AB
$*$	Push higher precedence	$+ *$	AB
C	Add operand	$+ *$	ABC
End	Pop all	Empty	$ABC * +$

$$A + B * C \longrightarrow ABC * +$$

Dry Run with Parentheses

Convert:

$$(A + B) * C$$

Symbol	Action	Stack	Postfix
(	Push	(	Empty
A	Add	(	A
+	Push	(+	A
B	Add	(+	AB
)	Pop to '('	Empty	AB+
*	Push	*	AB+
C	Add	*	AB + C
End	Pop all	Empty	AB + C*

$$(A + B) * C \longrightarrow AB + C*$$

Class Activity

Convert the following into postfix notation:

$$A * (B + C) - D$$

Class Activity

Convert the following into postfix notation:

$$A * (B + C) - D$$

Break the expression into parts:

$$(B + C) \longrightarrow BC +$$

$$A * (B + C) \longrightarrow ABC + *$$

$$A * (B + C) - D \longrightarrow ABC + * D -$$

$ABC + * D -$

Associativity Example

Convert:

$$A - B + C$$

Both '-' and '+' have equal precedence and are left-associative.

Therefore:

$$(A - B) + C$$

Postfix:

$$AB - C +$$

Associativity Example

Convert:

$$A - B + C$$

Both '-' and '+' have equal precedence and are left-associative.
Therefore:

$$(A - B) + C$$

Postfix:

$$AB - C +$$

It is not:

$$ABC + -$$

because that would represent:

Operator Helper Functions

```
1 int isOperator(char symbol)
2 {
3     return symbol == '+' ||
4            symbol == '-' ||
5            symbol == '*' ||
6            symbol == '/' ||
7            symbol == '%' ||
8            symbol == '^';
9 }
10
11 int precedence(char operator)
12 {
13     if (operator == '^')
14         return 3;
15
16     if (operator == '*' ||
17         operator == '/' ||
18         operator == '%')
19         return 2;
20
21     if (operator == '+' ||
22         operator == '-')
23         return 1;
```

Stack Operations for Conversion

```
1 #define MAX 100
2
3 char operatorStack[MAX];
4 int top = -1;
5
6 int pushOperator(char value)
7 {
8     if (top == MAX - 1)
9         return 0;
10
11     operatorStack[++top] = value;
12     return 1;
13 }
14
15 char popOperator(void)
16 {
17     if (top == -1)
18         return '\0';
19
20     return operatorStack[top--];
21 }
22 }
```

Infix-to-Postfix Function: Part 1

```
1 #include <ctype.h>
2 #include <string.h>
3
4 int infixToPostfix(
5     const char infix[],
6     char postfix[]
7 )
8 {
9     int i;
10    int outputIndex = 0;
11
12    top = -1;
13
14    for (i = 0; infix[i] != '\0'; i++)
15    {
16        char symbol = infix[i];
17
18        if (isspace((unsigned char)symbol))
19            continue;
20
21        if (isalnum((unsigned char)symbol))
22        {
```

Infix-to-Postfix Function: Part 2

```
1  else if (symbol == ')')
2  {
3      while (top != -1 &&
4             peekOperator() != '(')
5      {
6          postfix[outputIndex++] =
7              popOperator();
8      }
9
10     if (top == -1)
11         return 0;
12
13     popOperator();
14 }
15 else if (isOperator(symbol))
16 {
17     while (top != -1 &&
18            peekOperator() != '(' &&
19            precedence(peekOperator()) >=
20                precedence(symbol))
21     {
22         postfix[outputIndex++] =
```

Infix-to-Postfix Function: Part 3

```
1  while (top != -1)
2  {
3      if (peekOperator() == '(')
4          return 0;
5
6      postfix[outputIndex++] =
7          popOperator();
8  }
9
10 postfix[outputIndex] = '\0';
11 return 1;
12 }
```

Limitation

This introductory version treats every letter or digit as a single-character operand.

Program: Infix to Postfix

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char infix[MAX];
6     char postfix[MAX];
7
8     printf("Enter an infix expression: ");
9
10    if (fgets(infix, sizeof(infix), stdin)
11        == NULL)
12    {
13        printf("Input error\n");
14        return 1;
15    }
16
17    infix[strcspn(infix, "\n")] = '\0';
18
19    if (infixToPostfix(infix, postfix))
20    {
21        printf("Postfix expression: %s\n",
22            postfix);
23    }
```

Why is Postfix Easy to Evaluate?

Consider:

$$23 * 5 +$$

The evaluation order is unambiguous:

$$2 \times 3 = 6$$

$$6 + 5 = 11$$

No precedence table or parentheses are required.

Why is Postfix Easy to Evaluate?

Consider:

$$23 * 5 +$$

The evaluation order is unambiguous:

$$2 \times 3 = 6$$

$$6 + 5 = 11$$

No precedence table or parentheses are required.

Stack Usage

Operands and intermediate results are stored in an operand stack.

Postfix Evaluation Algorithm

Scan the postfix expression from left to right.

- ① If the token is an operand, push it.
- ② If the token is an operator:
 - Pop the second operand.
 - Pop the first operand.
 - Calculate:

first operand operator second operand
 - Push the result.
- ③ After scanning, the final stack value is the answer.

Operand Order is Critical

For the postfix expression:

82—

The first pop gives:

$$op_2 = 2$$

The second pop gives:

$$op_1 = 8$$

The operation is:

$$op_1 - op_2$$

$$8 - 2 = 6$$

Operand Order is Critical

For the postfix expression:

82—

The first pop gives:

$$op_2 = 2$$

The second pop gives:

$$op_1 = 8$$

The operation is:

$$op_1 - op_2$$

$$8 - 2 = 6$$

Dry Run: Postfix Evaluation

Evaluate:

9 3 4 * 8 + 4 / -

Token	Operand Stack
9	9
3	9, 3
4	9, 3, 4
*	9, 12
8	9, 12, 8
+	9, 20
4	9, 20, 4
/	9, 5
-	4

Result = 4

Evaluate:

$$62/3 - 4*$$

Interactive Evaluation

Evaluate:

$$62/3 - 4*$$

$$62 / \longrightarrow 3$$

$$33 - \longrightarrow 0$$

$$04 * \longrightarrow 0$$

0

Postfix Evaluation: Stack Functions

```
1 #define MAX_VALUES 100
2
3 double valueStack[MAX_VALUES];
4 int valueTop = -1;
5
6 int pushValue(double value)
7 {
8     if (valueTop == MAX_VALUES - 1)
9         return 0;
10
11     valueStack[++valueTop] = value;
12     return 1;
13 }
14
15 int popValue(double *value)
16 {
17     if (valueTop == -1)
18         return 0;
19
20     *value = valueStack[valueTop--];
21     return 1;
22 }
```

Postfix Evaluation Function

```
1 double evaluatePostfix(  
2     const char expression[],  
3     int *success  
4 )  
5 {  
6     int i;  
7     double first;  
8     double second;  
9     double result;  
  
10  
11     valueTop = -1;  
12     *success = 0;  
  
13  
14     for (i = 0; expression[i] != '\0'; i++)  
15     {  
16         char token = expression[i];  
  
17  
18         if (isspace((unsigned char)token))  
19             continue;  
  
20  
21         if (isdigit((unsigned char)token))  
22         {
```

Postfix Evaluation Function: Result

```
1  switch (token)
2  {
3  case '+':
4      result = first + second;
5      break;
6
7  case '-':
8      result = first - second;
9      break;
10
11 case '*':
12     result = first * second;
13     break;
14
15 case '/':
16     if (second == 0)
17         return 0.0;
18
19     result = first / second;
20     break;
21
22 case '%':
```

Recursion as a Stack Application

A recursive function is a function that calls itself to solve a smaller version of the same problem.

Every recursive solution must contain:

- 1 A base case
- 2 A recursive case

Recursion as a Stack Application

A recursive function is a function that calls itself to solve a smaller version of the same problem.

Every recursive solution must contain:

- 1 A base case
- 2 A recursive case

System Stack

Every recursive call creates an activation record containing parameters, local variables, and a return address.

Base Case and Recursive Case

Consider factorial:

$$n! = n(n - 1)!$$

Base case:

$$0! = 1$$

Recursive case:

$$n! = n(n - 1)! \quad \text{for } n > 0$$

Base Case and Recursive Case

Consider factorial:

$$n! = n(n - 1)!$$

Base case:

$$0! = 1$$

Recursive case:

$$n! = n(n - 1)! \quad \text{for } n > 0$$

Without a valid base case, recursive calls continue until the system stack is exhausted.

Stack overflow

Recursive Factorial

```
1 long long factorial(int n)
2 {
3     if (n == 0)
4         return 1;
5
6     return n * factorial(n - 1);
7 }
```

For:

factorial(4)

calls are created as:

factorial(4) → factorial(3) → factorial(2) → factorial(1) → factorial(0)

Factorial Call Stack

factorial(4): waiting for factorial(3)

factorial(3): waiting for factorial(2)

factorial(2): waiting for factorial(1)

factorial(1): waiting for factorial(0)

factorial(0): returns 1

The calls return in the reverse order:

1, 1, 2, 6, 24

Recursion and LIFO

Calls are made in this order:

$$F(4), F(3), F(2), F(1), F(0)$$

They return in this order:

$$F(0), F(1), F(2), F(3), F(4)$$

The last function called returns first.

LIFO

Therefore, recursion is an implicit application of the stack ADT.

Recursion Complexity

For recursive factorial:

$$T(n) = T(n - 1) + O(1)$$

Therefore:

$$T(n) = O(n)$$

Maximum recursion depth:

$$n + 1$$

Therefore, auxiliary stack space is:

$$O(n)$$

An iterative factorial also takes $O(n)$ time, but only $O(1)$ additional space.

Recursion vs Iteration

Feature	Recursion	Iteration
Control	Function calls	Loops
Memory	System stack	Usually constant
Code	Often shorter	Often more explicit
Overhead	Higher	Lower
Best use	Recursive problems	Repetitive processing

Recursion vs Iteration

Feature	Recursion	Iteration
Control	Function calls	Loops
Memory	System stack	Usually constant
Code	Often shorter	Often more explicit
Overhead	Higher	Lower
Best use	Recursive problems	Repetitive processing

Guideline

Use recursion when the problem has a natural recursive structure and the resulting solution is clearer.

Tower of Hanoi Problem

The problem consists of:

- Three poles: Source, Auxiliary, and Destination
- n disks of different sizes
- All disks initially placed on the source pole

Goal:

Move all disks to the destination pole

while preserving their size order.

Rules of Tower of Hanoi

- ① Move only one disk at a time.
- ② Only the top disk of a pole can be moved.
- ③ A larger disk cannot be placed above a smaller disk.

Rules of Tower of Hanoi

- ① Move only one disk at a time.
- ② Only the top disk of a pole can be moved.
- ③ A larger disk cannot be placed above a smaller disk.

For three disks, the minimum number of moves is:

7

Recursive Insight

To move n disks from pole A to pole C using pole B :

- 1 Move $n - 1$ disks from A to B using C .
- 2 Move disk n from A to C .
- 3 Move $n - 1$ disks from B to C using A .

Recursive Insight

To move n disks from pole A to pole C using pole B :

- 1 Move $n - 1$ disks from A to B using C .
- 2 Move disk n from A to C .
- 3 Move $n - 1$ disks from B to C using A .

Base case:

$$n = 1$$

Move the single disk directly from source to destination.

Tower of Hanoi Recurrence

Let $T(n)$ be the number of moves needed for n disks.

Two recursive subproblems require:

$$2T(n-1)$$

The largest disk requires one additional move.

Therefore:

$$T(n) = 2T(n-1) + 1$$

with:

$$T(1) = 1$$

Solving:

$$T(n) = 2^n - 1$$

Number of Moves

Number of Disks	Minimum Moves
1	1
2	3
3	7
4	15
5	31
10	1023
20	1,048,575

Number of Moves

Number of Disks	Minimum Moves
1	1
2	3
3	7
4	15
5	31
10	1023
20	1,048,575

The number of moves grows exponentially:

$$O(2^n)$$

Tower of Hanoi Function

```
1 void towerOfHanoi(  
2     int disks,  
3     char source,  
4     char destination,  
5     char auxiliary  
6 )  
7 {  
8     if (disks == 1)  
9     {  
10         printf(  
11             "Move disk 1 from %c to %c\n",  
12             source,  
13             destination  
14         );  
15  
16         return;  
17     }  
18  
19     towerOfHanoi(  
20         disks - 1,  
21         source,  
22         auxiliary,
```

Complete Tower of Hanoi Program

```
1 #include <stdio.h>
2
3 void towerOfHanoi(
4     int disks,
5     char source,
6     char destination,
7     char auxiliary
8 );
9
10 int main(void)
11 {
12     int disks;
13
14     printf("Enter number of disks: ");
15
16     if (scanf("%d", &disks) != 1 ||
17         disks <= 0)
18     {
19         printf("Invalid number of disks\n");
20         return 1;
21     }
22 }
```

Dry Run for Three Disks

Move three disks from A to C using B .

- 1 Move disk 1 from A to C
- 2 Move disk 2 from A to B
- 3 Move disk 1 from C to B
- 4 Move disk 3 from A to C
- 5 Move disk 1 from B to A
- 6 Move disk 2 from B to C
- 7 Move disk 1 from A to C

7 moves

Tower of Hanoi Complexity

Time recurrence:

$$T(n) = 2T(n-1) + 1$$

Therefore:

$$T(n) = O(2^n)$$

Maximum recursion depth:

$$n$$

Therefore, auxiliary space is:

$$O(n)$$

Tower of Hanoi Complexity

Time recurrence:

$$T(n) = 2T(n-1) + 1$$

Therefore:

$$T(n) = O(2^n)$$

Maximum recursion depth:

$$n$$

Therefore, auxiliary space is:

$$O(n)$$

Observation

The algorithm is compact and elegant, but its running time grows very rapidly.

Rapid-Fire Questions

- 1 Which symbols are directly copied during infix-to-postfix conversion?

Rapid-Fire Questions

- 1 Which symbols are directly copied during infix-to-postfix conversion?
- 2 Which stack is used during postfix evaluation?

Rapid-Fire Questions

- ❶ Which symbols are directly copied during infix-to-postfix conversion?
- ❷ Which stack is used during postfix evaluation?
- ❸ Which operand is popped first for a binary operator?

Rapid-Fire Questions

- ❶ Which symbols are directly copied during infix-to-postfix conversion?
- ❷ Which stack is used during postfix evaluation?
- ❸ Which operand is popped first for a binary operator?
- ❹ What are the two essential parts of recursion?

Rapid-Fire Questions

- ① Which symbols are directly copied during infix-to-postfix conversion?
- ② Which stack is used during postfix evaluation?
- ③ Which operand is popped first for a binary operator?
- ④ What are the two essential parts of recursion?
- ⑤ Why does recursion follow LIFO?

Rapid-Fire Questions

- 1 Which symbols are directly copied during infix-to-postfix conversion?
- 2 Which stack is used during postfix evaluation?
- 3 Which operand is popped first for a binary operator?
- 4 What are the two essential parts of recursion?
- 5 Why does recursion follow LIFO?
- 6 What is the minimum number of Tower of Hanoi moves?

Find the Postfix Expression

Convert:

$$(A + B) * (C - D)$$

- ① $AB + CD - *$
- ② $ABCD + -*$
- ③ $AB + *CD -$
- ④ $* + AB - CD$

Find the Postfix Expression

Convert:

$$(A + B) * (C - D)$$

① $AB + CD - *$

② $ABCD + -*$

③ $AB + *CD -$

④ $* + AB - CD$

$$(A + B) \longrightarrow AB +$$

$$(C - D) \longrightarrow CD -$$

$$AB + CD - *$$

Find the Evaluation Error

For the postfix expression:

84—

A student performs:

$$4 - 8 = -4$$

What is wrong?

Find the Evaluation Error

For the postfix expression:

$$84-$$

A student performs:

$$4 - 8 = -4$$

What is wrong?

The first popped value is the second operand:

$$op_2 = 4$$

The second popped value is the first operand:

$$op_1 = 8$$

Correct result:

Tower of Hanoi Question

How many moves are required for six disks?

$$T(n) = 2^n - 1$$

Tower of Hanoi Question

How many moves are required for six disks?

$$T(n) = 2^n - 1$$

$$T(6) = 2^6 - 1$$

$$= 64 - 1$$

63

Coding Exercises

- 1 Extend infix-to-postfix conversion to correctly process the right-associative exponent operator.
- 2 Modify postfix evaluation to support multi-digit operands separated by spaces.
- 3 Write a recursive function to calculate:

$$x^n$$

- 4 Count the number of function calls made by Tower of Hanoi.
- 5 Print the disk number with every Tower of Hanoi move.

Answer the following:

- 1 Convert:

$$A + B * (C - D)$$

into postfix form.

- 2 Evaluate:

$$52 + 3*$$

- 3 State the base case of recursive factorial.
- 4 How many moves are required for four Tower of Hanoi disks?
- 5 State the time and space complexity of Tower of Hanoi.

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Computer Science Press.
- ② Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
- ③ Reema Thareja, *Data Structures Using C*, Oxford University Press.

Questions?

Thank You